Automated Detection of Forged Banknotes using Machine Learning

Purpose of the Data Science Project

In this report, we shall describe how an unsupervised machine learning technique (namely k-means clustering) can be used to automate the detection of forged banknotes. The algorithm will automatically group the banknotes into 2 different groups: one of them will likely correspond to genuine notes and another will likely correspond to forged notes.

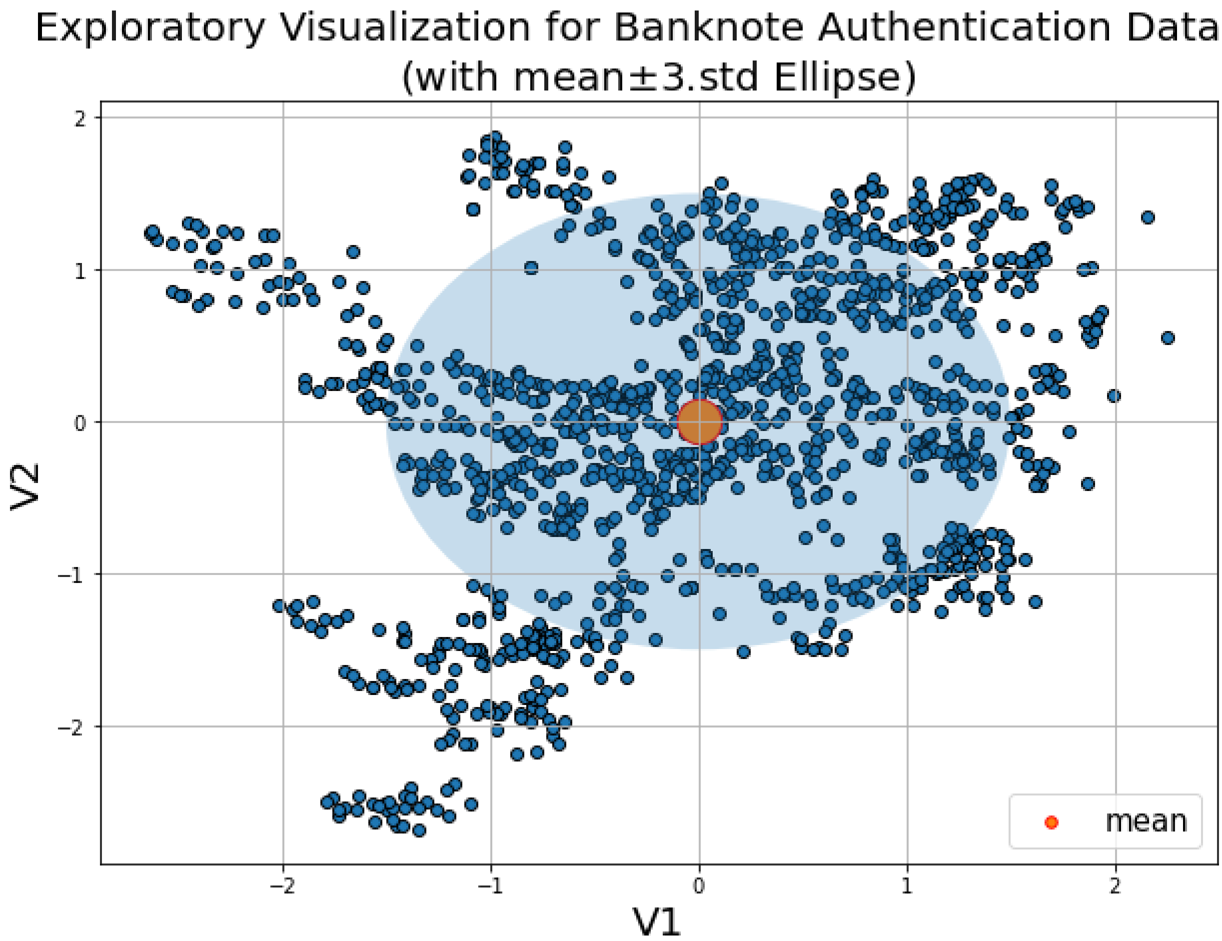
Description of the Dataset used

We shall use the Banknote authentication dataset to run the algorithm and obtain the automated grouping. We assume the dataset contains datapoints (examples) for both genuine and forged notes. Each datapoint in the dataset corresponds to couple of extracted features (namely, variance and skewness of the wavelet transform) from the image of a note. The following points describe the data briefly:

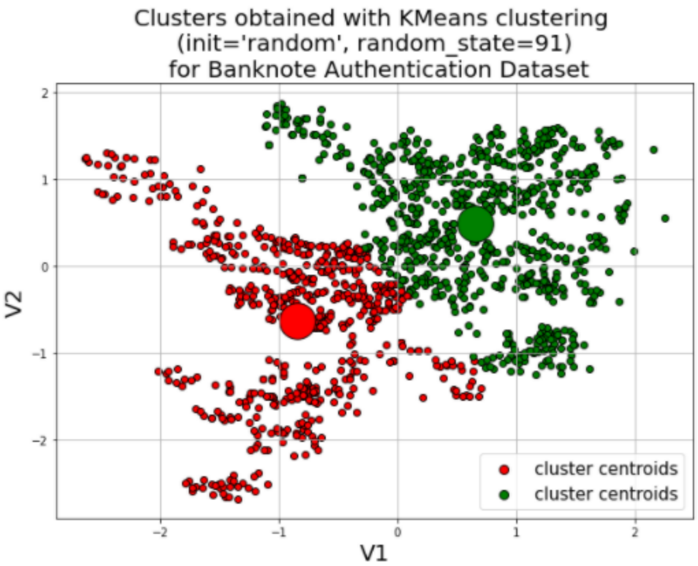
- The dataset has 1372 datapoints (rows) and 2 columns, namely V1 (variance) and V2 (skewness), the dataset is 2 dimensional.
- The mean value of the first feature (column) V1 is 0.433735. The standard deviation is 2.842763. The maximum and minimum values are 6.824800 and -7.042100, respectively, for this column.
- The mean value of the second feature (column) V2 is 1.922353. The standard deviation is 5.869047. The maximum and minimum values are 12.951600 and -13.773100, respectively, for this column.
- As can be seen, the mean of the variance column is smaller than that of skewness column, whereas the standard deviation for the skewness column is higher than the variance column.

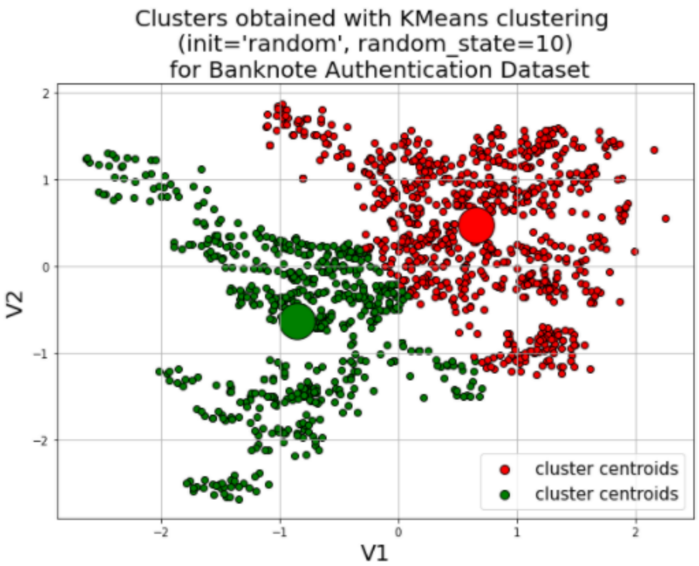
Data Analysis Methods

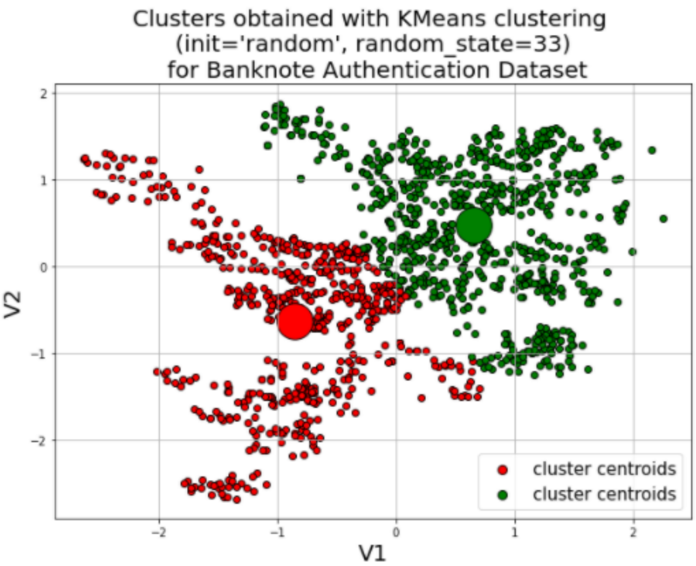
- We shall use k-means clustering algorithm to find 2 clusters from the bankotes dataset, expecting that one cluster will consist of mostly (if not all) genuine notes and another will consist of mostly (if not all) forged notes.
- But prior to applying the unsupervised machine learning model, we need to evaluate the dataset for its suitability for K-Means clustering.
- Hence, we shall start with some exploratory data analysis and visualization, along with some preprocessing needed of the data, prior to applying the k-means clustering algorithm.
- First notice that the 2-dimensional dataset has enough datapoints (namely 1372) to train the KMeans algorithm on.
- However, as can be seen from the previous section, the scale of the features V1 (variance) and V2 (skewness) are different. So a good idea will be to standardize (e.g., z-score normalize) the data prior to running KMeans on it, so that none of the features dominate over the other.
- Let's plot the 2-dimensional (normalized) dataset, along with the mean (centroid) point (colored red), with the uncertainty ellipse (characterized by mean±3.stdev). The following figure shows the plot. As we can see there are many datapoints outside the ellipse, those are likely the outliers in the dataset.

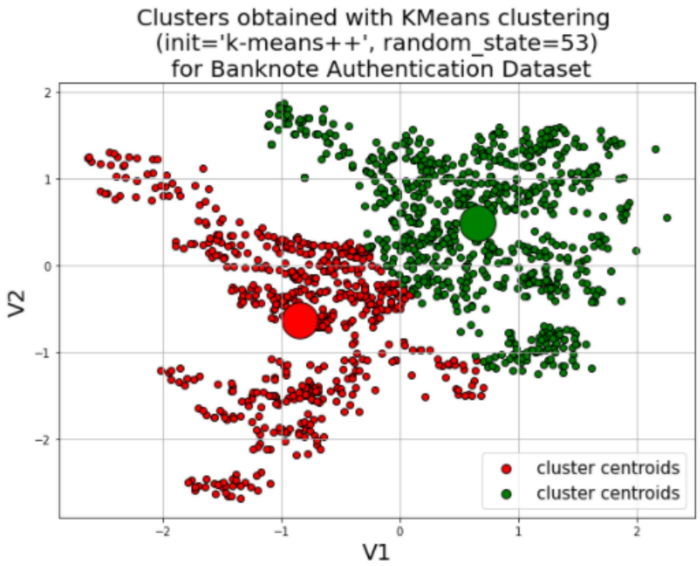


- Now that the dataset is ready to run the K-Means clustering on, we can run the clsutering algorithm to group the dataset into two clusters.
- Now, K-Means is a fast algorithm but it's sensitive to the initial centroids to start with and converges to local optimum. To ensure that the clusters obtained are stable, we need to re-run K-means with different initial centroids and visualize the resulting clusters obtained several times.
- The following figure shows the results obtained with different random initializations (and initial centroid selection using a more spohisticated *k-means*++).









- As can be seen from the figures, the result of the k-means clustering with different initializations (using different random states with scikit-learn) is pretty stable, final coordinates of the two cluster centroids are around:
  - (-0.8552756, -0.63006359)
  - (0.65494078, 0.48248113)
- K-Means clustering creates 2 clusters, one on left and the another one on right, with 597 points and 775 points, respectively.

Summary of Results

- Using the k-means algorithm, we can automatically cluster the datapoints into 2 groups, as shown in the above figure (one on left and the another one on right, with 597 points and 775 points, respectively).
- It's likely that one of the groups will consist of only / mostly geneuine notes and likewise the other will mostly / only consist of forged notes, thereby performing an automatic binary classifcation of the nodes.
- If we have ground-truth labels for the datapoints, we can actually confirm which group corresponds to forgred notes and compute the accuracy of the classification with k-means.

Recommendations

- It's good to have some datapoints (if not all) with ground-truth labels (notes with actual labels: genuine / forged) and use them to compute the accuracy of the automated clustering, before actually using the algorithm in production (if the desired accuracy is obtained).
- Use a separate (held-out) unseen test dataset (preferably with known labels) and use the k-means model to predict the labels, compute the accuracy on the test dataset, to prevent overfitting.
- If the model is underfitting, include some more features (e.g., Kurtosis) to increase training accuracy.
- As can be seen from the plots, the shape of the data is not spherical / circular and it is not ideal for KMeans clustering to find the clusters with very high accuracy (density based clustering or spectral clustering can do a better job here). Try other clustering algorithms too.
- If we have labelled dataset, we can use supervised (or semi-supervised, at least) classification algorithm to predict the label of a note.
- Finally, we can use a hybrid of different algorithms (models selected with cross-validation) to increase the accuracy of prediction.